Recurra

Team Build Guide: 5 members, 5 lanes, one repo

How to use this PDF

1. Open Claude Code in the repo folder (or a Claude chat).
2. Attach this PDF and type: "I am M2" (use your own member number).
3. Claude builds ONLY your lane, on your branch, and commits as you.
4. Push often. Open a Pull Request to main before 12:45 IST.

Feature freeze 12:45 IST | Submission 13:40 IST | 31 Oct 2026

0. Instructions for Claude (AI assistant) - read first

If you are Claude or another AI assistant and a team member has given you this PDF, follow these rules exactly. Five people are building one app in parallel against a hard deadline. Your job is to finish **one lane** fast and cleanly without breaking anyone else's work.

- **Identify the member.** The user will say "I am M1" ... "I am M5". If they have not said it, ask once: "Which member are you (M1-M5)?" and do nothing else until they answer.
- **Build only that member's lane** (sections 6 to 10). Do not build, fix or "improve" another lane's pages, even if it looks missing or broken. If another lane's feature is missing, show a placeholder link or skip it.
- **Respect file ownership** (section 4, Folder ownership). Never edit files outside the member's folders. Shared files belong to M1. If the lane truly needs a shared change, stop and tell the user: "Ask M1 to add ...".
- **Do not touch prisma/schema.prisma.** It is frozen. Use the models exactly as defined in section 5.
- **Check the foundation first** (M2-M5 only). Run `git pull origin main`. If src/lib/session.ts or prisma/schema.prisma is missing, tell the user that M1's foundation is not pushed yet, then prepare by reading the code style and planning your files. Do not create shared files yourself.
- **Git.** Work on the member's branch (given in their lane). Check `git config user.email` matches the member's GitHub email; if not, ask them to set it. Commit after every working piece with a clear message. Push to the branch at least every 20 minutes.
- **Priorities.** Each lane has P1 (must finish) and P2 (only if time is left). Finish and commit all P1 before starting P2. Stop new work at **12:45 IST**, then help the member open the Pull Request.
- **Quality bar** (these are judged): data from the DB only, Zod validation with clear inline errors, role checks on the server, responsive clean UI with shadcn, consistent navigation.
- **Explain briefly.** After each piece, tell the member in 2-3 lines what you built and how it works, so they can answer judges' questions about their own module.
- **Run and verify.** Run `npm run dev` and `npx tsc --noEmit` before each commit. Fix type errors in your own files only.

1. What we are building

Recurra is a Subscription Management System for Brightloop Software (the Odoo hackathon problem): products and variants, recurring plans, taxes, discounts, quotation templates, subscriptions with a status flow, automatic invoice generation from subscriptions, payments, a customer portal and revenue reports. Three roles: ADMIN (everything), INTERNAL (operations, no configuration) and PORTAL (customer, sees only their own data). Only ADMIN can create internal users and discounts.

Our highlights for judges: a Billing Run with dry-run preview and a "bill as of" date (time machine), a database unique constraint that makes double billing impossible (run it twice: 0 new invoices), GST split into CGST+SGST or IGST, an audit timeline per subscription, and a live dashboard with MRR, revenue, outstanding and overdue.

2. Team, lanes and timeline

Member	Lane	Branch	Main deliverables
M1	Lead: foundation and integration	main	Repo, setup, schema, shared libs, layout/sidebar, seed, merges, README, demo video
M2	Auth, users, dashboard, reports	feat/auth	Signup/login/reset with rules, admin creates staff, customers, KPI dashboard, reports with filters
M3	Catalog	feat/catalog	Taxes, products + variants, recurring plans, discounts (admin only)
M4	Sales	feat/subscriptions	Quotation templates, subscriptions + lines, status machine, audit timeline, customer portal
M5	Billing	feat/billing	Billing engine (dry run, commit, no double billing, GST), invoices + actions + print, payments

Time (IST)	What happens
11:10 - 11:30	M1 builds and pushes the foundation. M2-M5: install Node 20 + Git + Claude Code, set git identity, clone, read their lane.
11:30 - 12:45	Everyone pulls main, creates their branch, builds P1 then P2. Commit + push every 15-20 min.
12:45	FEATURE FREEZE. Each member: pull main, fix conflicts in own files, push, open PR.
12:45 - 13:15	M1 merges in order M2, M3, M4, M5. Reset + seed DB. Whole team tests the full flow (section 11).
13:15 - 13:40	README, demo video, submit.

3. Git workflow (everyone)

Contributions only count if commits use the email linked to **your** GitHub account.

```
# once per laptop
git config --global user.name  "Your Name"
git config --global user.email "your-github-email@example.com"

# M2-M5 after M1 pushes the foundation (~11:30)
git clone https://github.com/<M1-username>/recurra.git
cd recurra
npm install
cp .env.example .env
npx prisma migrate dev          # creates local SQLite DB
npx prisma db seed              # demo data
git checkout -b feat/<lane>    # feat/auth | feat/catalog | feat/subscriptions | feat/billing
npm run dev

# while working (every working piece)
git add -A
git commit -m "feat(<lane>): what you did"
git push -u origin feat/<lane>

# at 12:45
git fetch origin
git merge origin/main          # fix conflicts ONLY in your own files
git push
# then open a Pull Request on GitHub: feat/<lane> -> main
```

- M1 adds all 4 members as collaborators (GitHub repo, Settings, Collaborators) before 11:30.
- dev.db is in .gitignore: every laptop has its own local database (works offline).
- Never force-push. Never push directly to main (except M1 during foundation).

4. Shared conventions (this is CLAUDE.md)

M1 saves this as `CLAUDE.md` in the repo root. Claude Code reads it automatically on every machine.

```
# Recurra - Subscription Management System (Odoo Hackathon)

## Stack
Next.js App Router + TypeScript, Tailwind + shadcn/ui, Prisma + SQLite, Zod, bcryptjs, jose.

## Hard rules
- Build ONLY the lane of the member you are working for (M1..M5). See the build guide PDF.
- NEVER edit files owned by another lane. Shared files are owned by M1:
  prisma/*, src/lib/*, src/middleware.ts, src/components/app-sidebar.tsx,
  src/components/ui/*, src/app/layout.tsx, src/app/(app)/layout.tsx, src/app/page.tsx,
  CLAUDE.md, README.md, package.json.
- Need a shared change? Stop and tell the user to ask M1. Do not work around it.
- Need a new npm package? Ask the user first (package.json conflicts).
- All data comes from the DB through Prisma. No hardcoded or static JSON data.
- Every form: Zod validation in the Server Action + inline error messages in the UI.
- Every Server Action and page calls requireRole([...]) from @/lib/session.
- UI: shadcn components, p-6 page padding, gap-4, status shown with <Badge>,
  tables wrapped in overflow-x-auto (mobile friendly), toast on success/error.
- Money: round2() and formatINR() from @/lib/money. Dates: addDays/addPeriod from @/lib/dates.
- Commit small and often with clear messages, e.g. "feat(catalog): add plan form".

## Shared helpers (made by M1)
import { prisma } from "@/lib/db"
import { getSession, requireRole, createSession, destroySession } from "@/lib/session"
  getSession(): Promise<{ userId: string; role: "ADMIN"|"INTERNAL"|"PORTAL"; name: string } | null>
  requireRole(roles: Role[]): Promise<Session>   // redirects to /login if not allowed
import { round2, formatINR } from "@/lib/money"
import { addDays, addPeriod } from "@/lib/dates"
  // addPeriod(date, "DAILY"|"WEEKLY"|"MONTHLY"|"YEARLY")

## Folder ownership
M1: shared files above, prisma/seed.ts
M2: src/app/(auth)/**, src/app/(app)/users/**, customers/**, dashboard/**, reports/**,
    src/features/auth/**, src/features/reports/**
M3: src/app/(app)/products/**, taxes/**, plans/**, discounts/**, src/features/catalog/**
M4: src/app/(app)/templates/**, subscriptions/**, src/app/portal/**, src/features/subscriptions/**
M5: src/app/(app)/billing/**, invoices/**, payments/**, src/features/billing/**

## Status rules
Subscription: DRAFT -> QUOTATION -> CONFIRMED -> ACTIVE -> CLOSED ; ACTIVE <-> PAUSED (if plan.pausable)
Invoice: DRAFT -> CONFIRMED -> PAID ; DRAFT or CONFIRMED -> CANCELLED

## Demo accounts (seed)
admin@recurra.in / Admin@1234     staff@recurra.in / Staff@1234     customer1@demo.in / Customer@1234
```

5. Database schema (frozen - M1 creates, nobody edits)

File: `prisma/schema.prisma`. Enum-like fields are Strings (SQLite); allowed values are in the comments. `.env`: `DATABASE_URL="file:./dev.db"` and `JWT_SECRET="any-long-random-string"`.

```
generator client {
  provider = "prisma-client-js"
}

datasource db {
  provider = "sqlite"
  url      = env("DATABASE_URL")
}

// role: ADMIN | INTERNAL | PORTAL
model User {
  id               String         @id @default(cuid())
  name             String
  email            String         @unique
  passwordHash     String
  role             String         @default("PORTAL")
  phone            String?
  address          String?
  state            String?
  gstin            String?
  resetToken       String?
  resetTokenExpiry DateTime?
  createdAt        DateTime       @default(now())
  subscriptions    Subscription[]
  invoices         Invoice[]
}

model Tax {
  id        String             @id @default(cuid())
  name      String
  percent   Float
  products  Product[]
  lines     SubscriptionLine[]
  createdAt DateTime           @default(now())
}

// type: SERVICE | GOODS
model Product {
  id                String             @id @default(cuid())
  name              String
  type              String             @default("SERVICE")
  salesPrice        Float
  costPrice         Float
  notes             String?
  active            Boolean            @default(true)
  taxId             String?
  tax               Tax?               @relation(fields: [taxId], references: [id])
  variants          ProductVariant[]
  subscriptionLines SubscriptionLine[]
  templateLines     TemplateLine[]
  discounts         DiscountProduct[]
  createdAt         DateTime           @default(now())
}

model ProductVariant {
  id         String             @id @default(cuid())
  productId  String
  product    Product            @relation(fields: [productId], references: [id], onDelete: Cascade)
  attribute  String
  value      String
  extraPrice Float              @default(0)
  lines      SubscriptionLine[]
}

// billingPeriod: DAILY | WEEKLY | MONTHLY | YEARLY
model RecurringPlan {
  id            String              @id @default(cuid())
  name          String
  price         Float
  billingPeriod String
  minQuantity   Int                 @default(1)
  startDate     DateTime
  endDate       DateTime?
```

```
  autoClose     Boolean             @default(false)
  closable      Boolean             @default(true)
  pausable      Boolean             @default(false)
  renewable     Boolean             @default(true)
  subscriptions Subscription[]
  templates     QuotationTemplate[]
  createdAt     DateTime            @default(now())
}

// type: FIXED | PERCENT ; appliesTo: PRODUCT | SUBSCRIPTION
model Discount {
  id            String            @id @default(cuid())
  name          String
  type          String
  value         Float
  minPurchase   Float             @default(0)
  minQuantity   Int               @default(1)
  startDate     DateTime
  endDate       DateTime
  usageLimit    Int?
  usedCount     Int               @default(0)
  appliesTo     String            @default("SUBSCRIPTION")
  products      DiscountProduct[]
  subscriptions Subscription[]
  createdAt     DateTime          @default(now())
}

model DiscountProduct {
  discountId String
  productId  String
  discount   Discount @relation(fields: [discountId], references: [id], onDelete: Cascade)
  product    Product  @relation(fields: [productId], references: [id], onDelete: Cascade)

  @@id([discountId, productId])
}

model QuotationTemplate {
  id           String         @id @default(cuid())
  name         String
  validityDays Int
  planId       String
  plan         RecurringPlan  @relation(fields: [planId], references: [id])
  lines        TemplateLine[]
  createdAt    DateTime       @default(now())
}

model TemplateLine {
  id         String            @id @default(cuid())
  templateId String
  template   QuotationTemplate @relation(fields: [templateId], references: [id], onDelete: Cascade)
  productId  String
  product    Product           @relation(fields: [productId], references: [id])
  quantity   Int               @default(1)
}

// status: DRAFT | QUOTATION | CONFIRMED | ACTIVE | PAUSED | CLOSED
model Subscription {
  id              String             @id @default(cuid())
  number          String             @unique
  customerId      String
  customer        User               @relation(fields: [customerId], references: [id])
  planId          String
  plan            RecurringPlan      @relation(fields: [planId], references: [id])
  startDate       DateTime
  expirationDate  DateTime?
  paymentTerms    String             @default("Immediate")
  status          String             @default("DRAFT")
  nextBillingDate DateTime?
  quoteValidUntil DateTime?
  discountId      String?
  discount        Discount?          @relation(fields: [discountId], references: [id])
  lines           SubscriptionLine[]
  invoices        Invoice[]
  logs            AuditLog[]
  createdAt       DateTime           @default(now())
  updatedAt       DateTime           @updatedAt

  @@index([status, nextBillingDate])
}
```

```
model SubscriptionLine {
  id             String          @id @default(cuid())
  subscriptionId String
  subscription   Subscription    @relation(fields: [subscriptionId], references: [id], onDelete: Cascade)
  productId      String
  product        Product         @relation(fields: [productId], references: [id])
  variantId      String?
  variant        ProductVariant? @relation(fields: [variantId], references: [id])
  taxId          String?
  tax            Tax?            @relation(fields: [taxId], references: [id])
  quantity       Int
  unitPrice      Float
  amount         Float
}

// status: DRAFT | CONFIRMED | PAID | CANCELLED
model Invoice {
  id             String        @id @default(cuid())
  number         String        @unique
  subscriptionId String
  subscription   Subscription  @relation(fields: [subscriptionId], references: [id])
  customerId     String
  customer       User          @relation(fields: [customerId], references: [id])
  periodStart    DateTime
  periodEnd      DateTime
  dueDate        DateTime
  subtotal       Float
  discountTotal  Float         @default(0)
  taxTotal       Float
  total          Float
  amountPaid     Float         @default(0)
  status         String        @default("DRAFT")
  lines          InvoiceLine[]
  payments       Payment[]
  createdAt      DateTime      @default(now())

  @@unique([subscriptionId, periodStart])
  @@index([status, dueDate])
}

model InvoiceLine {
  id          String  @id @default(cuid())
  invoiceId   String
  invoice     Invoice @relation(fields: [invoiceId], references: [id], onDelete: Cascade)
  description String
  quantity    Int
  unitPrice   Float
  taxPercent  Float   @default(0)
  discount    Float   @default(0)
  amount      Float
}

// method: UPI | CARD | BANK | CASH
model Payment {
  id          String   @id @default(cuid())
  invoiceId   String
  invoice     Invoice  @relation(fields: [invoiceId], references: [id])
  method      String
  amount      Float
  paymentDate DateTime
  note        String?
  createdAt   DateTime @default(now())
}

model AuditLog {
  id             String        @id @default(cuid())
  entity         String
  entityId       String
  action         String
  details        String?
  actorId        String?
  subscriptionId String?
  subscription   Subscription? @relation(fields: [subscriptionId], references: [id])
  createdAt      DateTime      @default(now())
}
```

6. M1 - Lead: Foundation and Integration

Branch	Owns (only edit these)
main (foundation), then reviews PRs	prisma/*, src/lib/*, src/middleware.ts, src/components/app-sidebar.tsx, src/components/ui/*, src/app/layout.tsx, src/app/(app)/layout.tsx, src/app/page.tsx, placeholder page.tsx files, CLAUDE.md, README.md, package.json, .env.example

P1 - must finish

- **By 11:30:** create the repo and run the setup commands below; schema from section 5; CLAUDE.md from section 4.
- Shared libs: db.ts, session.ts, money.ts, dates.ts with exactly the signatures in CLAUDE.md.
- Layout with responsive sidebar (links filtered by role) and a placeholder page.tsx for every route, so links work before other lanes merge.
- middleware.ts, root page.tsx (redirects by role), seed.ts with demo data, .env.example. Push to main and tell the team.
- **After 11:30:** add collaborators, answer shared-file requests from M2-M5, review PRs, write README.

P2 - only if P1 is committed and time is left

- Record the demo video (section 11). Prepare 3 slides if the hackathon asks for a pitch.

Prompt to give Claude

I am M1 (Lead). Read CLAUDE.md and the build guide. In the current folder run: `npx create-next-app@latest . --typescript --tailwind --eslint --app --src-dir --import-alias "@/*" --use-npm`, then `npm i prisma @prisma/client bcryptjs zod jose`, `npm i -D @types/bcryptjs tsx`, `npx shadcn@latest init -d` and `npx shadcn@latest add button input label card table badge dialog select textarea tabs sonner dropdown-menu checkbox switch`. Write prisma/schema.prisma exactly as in section 5 and CLAUDE.md exactly as in section 4. Create src/lib/db.ts (Prisma singleton), src/lib/session.ts (JWT in httpOnly cookie 'session' via jose and JWT_SECRET; createSession(user), getSession(), destroySession(), requireRole(roles) that redirects to /login), src/lib/money.ts (round2, formatINR with en-IN and INR), src/lib/dates.ts (addDays, addPeriod for DAILY/WEEKLY/MONTHLY/YEARLY). Create src/app/(app)/layout.tsx with a responsive collapsible sidebar (src/components/app-sidebar.tsx) grouped Overview (Dashboard, Reports), Catalog (Products, Taxes, Plans, Discounts), Sales (Customers, Templates, Subscriptions), Billing (Billing Run, Invoices, Payments), Admin (Users); hide Users/Discounts from non-admins; show the user's name, role and a Logout button. Add Toaster to the root layout. Create a simple placeholder page.tsx for every route in CLAUDE.md, plus src/app/portal/page.tsx and src/app/(auth)/login/page.tsx placeholders. Add src/middleware.ts redirecting to /login when the session cookie is missing (allow /login, /signup, /forgot-password, /reset). Root page.tsx redirects ADMIN/INTERNAL to /dashboard and PORTAL to /portal. Write prisma/seed.ts using bcrypt with the demo accounts from CLAUDE.md, 3 PORTAL customers (2 in Kerala, 1 in Karnataka), GST 18% tax, 2 products (one with variant Brand=Odoo +560), Monthly and Yearly plans, one active 10% discount; register the seed in package.json ("prisma": {"seed": "tsx prisma/seed.ts"}). Add dev.db to .gitignore and create .env.example. Run migrate + seed + npm run dev, make sure it compiles, then commit and push to main.

Done when

- Fresh clone + npm install + migrate + seed + npm run dev works.
- Every sidebar link opens a page; admin login placeholder exists; schema matches section 5 exactly.

7. M2 - Auth, Users, Dashboard, Reports

Branch	Owns (only edit these)
feat/auth	src/app/(auth)/** (login, signup, forgot-password, reset/[token]), src/app/(app)/users/**, customers/**, dashboard/**, reports/**, src/features/auth/**, src/features/reports/**

P1 - must finish

- /signup: name, email, password, confirm password. Zod: valid and UNIQUE email (check DB), password longer than 8 chars with uppercase, lowercase and a special character; show the rules as a live checklist. New user role = PORTAL.
- /login: bcrypt compare, createSession, redirect ADMIN/INTERNAL to /dashboard, PORTAL to /portal. Clear error on wrong credentials.
- /forgot-password: create random resetToken (stored hashed or plain) with 30 min expiry. Offline mode: show the reset link on screen and console.log it. /reset/[token]: check expiry, enforce password rules, clear token after use.
- /users (ADMIN only, server-enforced): list all users with role badges; form to create INTERNAL users. Only admin can do this.
- /customers (ADMIN, INTERNAL): list, search, create, edit PORTAL customers (name, email, phone, address, state, GSTIN with format check).

P2 - only if P1 is committed and time is left

- /dashboard: KPI cards - active subscriptions, revenue (sum of payments), outstanding (sum of total - amountPaid on CONFIRMED invoices), overdue invoices (CONFIRMED, dueDate < today, not fully paid), MRR (monthly-normalised plan price x qty of ACTIVE subs). Recent invoices table.
- /reports: same metrics with filters (date range, plan, subscription status), overdue invoice table, payments table. Use Prisma aggregate/groupBy, not JS loops over everything.

Prompt to give Claude

I am M2. Read CLAUDE.md and section 7 of the build guide. Work only on branch feat/auth and only in the folders listed for M2. Build all P1 items in order, committing after each one, then the P2 items. Use Server Actions with Zod, shadcn forms with inline errors, requireRole on every page and action, and createSession/destroySession from @/lib/session. If the dashboard needs invoice or payment data that does not exist yet, the queries should still work and show zeros.

Done when

- Weak password is rejected with clear messages; duplicate email is rejected.
- Login redirects by role; INTERNAL user cannot open /users even by typing the URL.
- Reset flow works end to end; dashboard and reports load real numbers from the DB.

8. M3 - Catalog: Taxes, Products, Variants, Plans, Discounts

Branch	Owns (only edit these)
feat/catalog	src/app/(app)/products/**, taxes/**, plans/**, discounts/**, src/features/catalog/**

P1 - must finish

- /taxes: CRUD; name, percent between 0 and 100. ADMIN edits, INTERNAL view only.
- /products: list with search; create/edit with name, type (SERVICE/GOODS), salesPrice and costPrice (>= 0), notes, tax select. Product detail page with a Variants section: add/remove variant (attribute, value, extraPrice), e.g. Brand / Odoo / 560. Show final price = salesPrice + extraPrice per variant.
- /plans: CRUD; name, price, billingPeriod select (DAILY/WEEKLY/MONTHLY/YEARLY), minQuantity >= 1, startDate, endDate (must be after startDate), switches for autoClose, closable, pausable, renewable shown as badges in the list.
- /discounts (ADMIN only, enforced on server and hidden from INTERNAL): name, type FIXED/PERCENT (percent <= 100), value, minPurchase, minQuantity, startDate/endDate, usageLimit, appliesTo PRODUCT/SUBSCRIPTION; when PRODUCT, multi-select products (DiscountProduct). List shows usedCount / usageLimit and Active / Scheduled / Expired badge.

P2 - only if P1 is committed and time is left

- Delete confirmation dialogs; block deleting a product or plan that is used by a subscription (friendly error).
- Empty states with a "Create first ..." button on every list.

Prompt to give Claude

I am M3. Read CLAUDE.md and section 8 of the build guide. Work only on branch feat/catalog and only in the folders listed for M3. Build all P1 items in order (taxes, products + variants, plans, discounts), committing after each one, then P2. Use Server Actions with Zod, shadcn forms and tables with inline errors and toasts, and requireRole on every page and action (ADMIN can edit everything; INTERNAL can only view taxes, products and plans; discounts are ADMIN only).

Done when

- Admin can create the demo setup: GST 18%, a product with Odoo variant +560, a monthly plan, a 10% discount.
- Invalid input (negative price, end before start, percent > 100) shows inline errors.
- INTERNAL user cannot create/edit catalog items or open /discounts.

9. M4 - Templates, Subscriptions, Status Flow, Portal

Branch	Owns (only edit these)
feat/subscriptions	src/app/(app)/templates/**, subscriptions/**, src/app/portal/**, src/features/subscriptions/**

P1 - must finish

- /templates: CRUD; name, validityDays (>= 1), recurring plan, product lines (product + quantity).
- /subscriptions list: status badges, filter by status, search by number or customer name.
- Create form: customer (PORTAL users), plan, startDate, expirationDate (after start), paymentTerms, optional discount, optional "Start from template" that prefills plan and lines. Lines: product, optional variant, quantity (>= plan.minQuantity), unitPrice = salesPrice + variant.extraPrice (editable), tax (default product tax), amount = qty x unitPrice computed live. Number auto-generated SUB-0001, SUB-0002 ...
- src/features/subscriptions/status.ts: allowed-transitions map. DRAFT->QUOTATION, QUOTATION->CONFIRMED or DRAFT, CONFIRMED->ACTIVE, ACTIVE->PAUSED (only if plan.pausable) or CLOSED (only if plan.closable), PAUSED->ACTIVE or CLOSED. Enforced in a Server Action; the UI shows only valid action buttons.
- On QUOTATION set quoteValidUntil = today + validityDays (default 30); block confirming an expired quotation. On ACTIVE set nextBillingDate = startDate (this is what M5's billing engine reads).
- Write an AuditLog row (entity 'Subscription', subscriptionId set) for create, edit and every status change; show a vertical Timeline on the subscription detail page. Detail page also lists the subscription's invoices (read-only).

P2 - only if P1 is committed and time is left

- /portal (PORTAL only): my subscriptions and my invoices, filtered by session userId on the server. Customer can accept a QUOTATION (-> CONFIRMED) and pause / close if the plan allows. Invoice links open a read-only view.
- Edit lines only while DRAFT or QUOTATION.

Prompt to give Claude

I am M4. Read CLAUDE.md and section 9 of the build guide. Work only on branch feat/subscriptions and only in the folders listed for M4. Build all P1 items in order, committing after each one, then P2. Use Server Actions with Zod, shadcn forms with inline errors, requireRole on every page and action (ADMIN and INTERNAL manage subscriptions; PORTAL only uses /portal). Keep the status transition map in one file and enforce it on the server.

Done when

- A subscription created from a template goes DRAFT -> QUOTATION -> CONFIRMED -> ACTIVE with a timeline entry for each step.
- An invalid jump (e.g. DRAFT -> ACTIVE) is rejected by the server; pause only appears when the plan is pausable.
- ACTIVE subscriptions have nextBillingDate set; a customer only sees their own data in /portal.

10. M5 - Billing Engine, Invoices, Payments

Branch	Owns (only edit these)
feat/billing	src/app/(app)/billing/**, invoices/**, payments/**, src/features/billing/**

P1 - must finish

- src/features/billing/engine.ts: runBilling(asOfDate, mode 'DRY' | 'COMMIT'). Find subscriptions with status ACTIVE and nextBillingDate <= asOfDate. periodStart = nextBillingDate; periodEnd = addPeriod(periodStart, plan.billingPeriod) - 1 day. Invoice lines from subscription lines.
- Discount: apply the subscription's discount only if asOfDate is within start/end, usedCount < usageLimit (or no limit), total quantity >= minQuantity and subtotal >= minPurchase. FIXED = min(value, subtotal); PERCENT = subtotal x value / 100.
- Tax per line from line tax percent on the discounted amount. GST split for display: customer.state == 'Kerala' (company state) -> CGST and SGST = half each, otherwise IGST. Store taxTotal on the invoice.
- COMMIT inside prisma.$transaction per subscription: create Invoice (number INV-0001..., status CONFIRMED, dueDate = periodStart + 7 days), advance nextBillingDate, increment discount usedCount, write AuditLog. Catch the unique-constraint error on (subscriptionId, periodStart) and count it as 'already billed - skipped'. If past expirationDate and plan.autoClose, set CLOSED.
- Return summary: invoices created, total amount, discounts applied, skipped list with reason.
- /billing (ADMIN, INTERNAL; COMMIT for ADMIN only): "Bill as of" date picker (time machine), Preview button shows the DRY result table, Run button commits and shows the summary. Running twice must create 0 new invoices.
- /invoices: list with status filter and an Overdue badge (CONFIRMED, dueDate < today, amountPaid < total). Detail page: customer details, lines, discount, CGST/SGST or IGST, totals, outstanding. Actions per status rules: Confirm, Cancel, Send, Print.

P2 - only if P1 is committed and time is left

- Send: mark as sent in AuditLog and show a toast (offline, no email). Print: clean print view with print CSS and window.print().
- Record Payment on the invoice page: method (UPI/CARD/BANK/CASH), amount (> 0 and <= outstanding), date, note. Update amountPaid; set PAID when fully paid. /payments lists all payments with filters.
- Manual "Create invoice" button on a subscription is NOT needed - billing run covers it.

Prompt to give Claude

I am M5. Read CLAUDE.md and section 10 of the build guide. Work only on branch feat/billing and only in the folders listed for M5. Build all P1 items in order, committing after each one, then P2. The billing engine is the most important logic in the project: keep it in src/features/billing/engine.ts as pure, well-commented functions, use prisma.$transaction, and rely on the @@unique([subscriptionId, periodStart]) constraint for idempotency. If there are no ACTIVE subscriptions yet (M4 not merged), create test ones in a local script under src/features/billing/dev-seed.ts (do not edit prisma/seed.ts).

Done when

- Preview shows what will be billed; Run creates invoices; Run again creates 0 and lists them as skipped.
- Invoice shows correct discount and GST split; overdue badge appears for a past due date.
- Recording payments moves the invoice to PAID and the outstanding amount to 0.

11. Integration, testing and demo (whole team, 12:45 onward)

Merge order (M1)

- Merge PRs in order: feat/auth, feat/catalog, feat/subscriptions, feat/billing. After each merge run `npm run build` on main.
- Then `npx prisma migrate reset` (re-creates DB and runs seed) and test the flow below.
- Bug in a lane? The owner fixes it on their branch and opens a small follow-up PR, so the fix is credited to them.

Full test / demo flow (about 3 minutes)

#	Who / role	Step	Lane
1	Visitor	Sign up with a weak password (errors shown), then a strong one; lands in the portal	M2
2	Admin	Create GST 18%, product + variant Brand=Odoo +560, monthly plan, 10% discount	M3
3	Admin	Create an internal user; show that staff cannot open Users or Discounts	M2
4	Staff	Create subscription from template; Draft -> Quotation -> Confirmed -> Active; show the timeline	M4
5	Admin	Billing: preview, run, run again (0 new, all skipped) - no double billing	M5
6	Staff	Open invoice: discount + CGST/SGST; print it; record payment -> PAID	M5
7	Admin	Set 'Bill as of' 2 months ahead: new invoices, an overdue one appears	M5
8	Customer	Portal: sees only their subscriptions and invoices; pause if allowed	M4
9	Admin	Dashboard + reports: active subs, revenue, outstanding, overdue, MRR, filters	M2

Submission checklist

- Every required module works with real DB data (no static JSON).
- Validation errors are visible on every form; UI is responsive with consistent colors and navigation.
- GitHub shows commits from all 5 members; each lane merged via its own PR.
- README: problem, features, tech stack, setup steps (clone, install, migrate, seed, dev), demo accounts, screenshots.
- Runs fully offline on a laptop (local SQLite, no cloud services).
- Every member can explain their own module in 30 seconds.

Pitch line: Brightloop doesn't need a better spreadsheet. It needs billing that runs itself. Recurra bills on schedule, can never bill twice, applies taxes and discounts on its own, and shows finance the real numbers on one screen.